Undertow - Step 2 complete: the live feed

Built and tested on real Monad mainnet data, 2026-10-09 20:45:58 UTC (block 111,994,834).

What this step delivers

Step 1 could read Perpl once. Step 2 keeps that picture fresh, remembers how it changes, and says who each account is. It also settles a design question that decides how 'live' the product can honestly claim to be, by measuring how fast each way of reading Perpl really is.

Refresh cycle	Accounts mapped to wallet address	History rows saved	Open-interest checks	Tests
73.4 s	434 of 434	55 (5 snapshots)	16 / 16 pass	13 / 13 pass

What was built

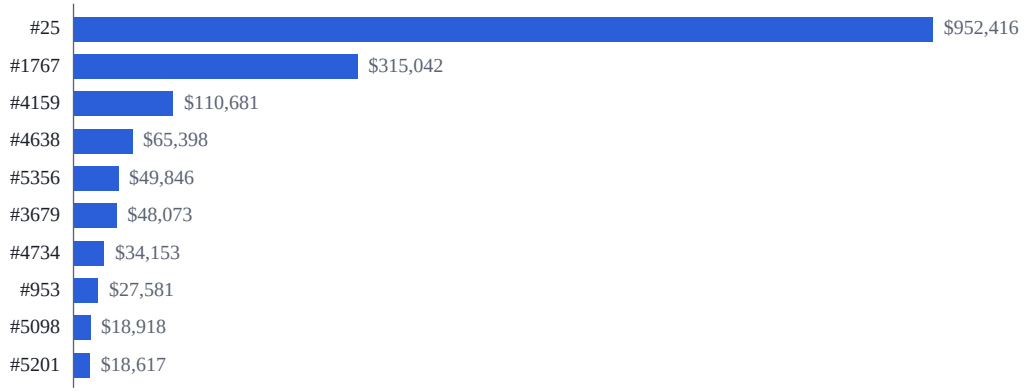
Piece	What it does
`refresher.py`	Loop that takes a full snapshot, parses and checks it, writes the latest file atomically (a screen never reads a half-written file), appends one row per market to a history file, and writes a status file with timings. Accepts your own RPC address (for example Alchemy).
`onchain.py`	Reads Perpl's contract directly with batched requests, using only Python's standard library. Retries politely when the public server says 'too many requests'.
`accounts.py`	Registry of account number, wallet address and balance, cached on disk and refreshed hourly. This is what the Wallet explorer needs to look up a trader by address.
`blocks.py`	Decodes the raw events inside a block (trades, fills, orders, liquidations) and lists which accounts a block touched.
`abi/Exchange.json`	Perpl's contract definition, copied from the official SDK, ready for the Envio indexer.

Measurements that decided the design

Way of reading Perpl	Measured speed	Verdict
Full snapshot (every account, every order)	33 to 48 s	**Used.** Fast enough for a screen refreshed under a minute.
Replay one block's events	about 1.5 s per block	Not used for state. Monad makes a block every 0.4 s, so replay falls behind.
`show account` for one trader	about 11 s	Too slow for lists. Kept only for one-wallet detail.
Direct contract read, 10 accounts per request	432 accounts in 41 s	**Used.** Replaces roughly 80 minutes of CLI calls.

What the new data shows

Balances of the accounts that currently hold positions (AUSD, the deposit currency). This is deposit size, not position size.



Findings

- **Money is concentrated.** The 434 accounts with positions hold $1,815,349 in total. The largest, account #25, holds $952,416, about 52% of it. For a liquidation map this means one account's choices can move the picture, which fits the 'big zones are visible targets' story but also means we must label whale-driven zones honestly.
- **Address lookup works.** Every account with a position now has a wallet address. Spot checks on four accounts matched Perpl's own CLI exactly (address and balance). Wallet explorer search by address is unblocked.
- **A full refresh needs no special connection.** About 45 seconds on the free public server. An Alchemy key would cut this sharply because it removes the 15-requests-per-second limit, but nothing is blocked without it.

Important discovery for the cascade simulator

The roadmap describes a liquidation as forced selling into the order book. Perpl's contract is more complex. Its events include `PositionLiquidated` (with an `onOrderBook` flag), `PositionDeleveraged` (auto-deleveraging against opposing positions) and a `BuyToLiquidate` process in which a buyer takes over a position, with a slippage limit. So a liquidated position does not always dump into the book as a market sale.

Consequence: in Step 3 I will read the SDK's liquidation logic before writing the cascade simulator, and the simulator will say plainly which liquidations it assumes hit the book. If that is only partly true, the pitch wording ('forced selling pushes the price down') must be softened to match. A claim the contract does not support would hurt the 'trustworthy risk screen' that Track 1 judges look for.

How to run it

```
python -m undertow.refresher --once
python -m undertow.refresher --pause 5
python -m undertow.refresher --rpc https://YOUR-ALCHEMY-URL --pause 0
python -m unittest discover -s tests
```

Honest limits and open items

- **Fresh to about a minute, not to the block.** The data is a rolling snapshot, so the screen should say 'updated N seconds ago'. The roadmap's 'about 0.4 seconds' claim for liquidation-aware orders is not yet supported. Closing that gap needs an event feed on a faster connection, planned for Step 9.
- **The Envio indexer is not built.** Docker and the Envio tool are not installed on this machine, so I could not test one. The contract definition is saved and ready. This needs a decision from you: install Docker Desktop and Envio, use Envio's hosted option, or fall back to our own history file for charts. The Envio prize ($1,000) requires a real indexer.
- **No Alchemy key yet.** Optional. It would speed up everything, and the Alchemy sponsor offers $1,000 in credits.
- **Liquidation events are decoded but untested on a real liquidation.** The block I captured had none. The event names come from the contract definition and a unit test covers name matching, but I have not seen a real one parsed yet.
- **History has only just started.** 5 snapshots so far, so there is nothing to chart over days yet. Leave the refresher running to build it. 24-hour volume needs trade history, which is the Envio job.
- **Public server limits.** It rejects large request bursts (we hit this and added backoff). Heavy use needs your own RPC.

What is left

Step	Scope	Status
3	Risk engine: read SDK liquidation rules, verify our formula against Perpl's liquidation prices, zones, cascade simulator	Next
4	Radar map and plain-language sentence	Not started
5	Protocol view (volume, deposits, fees, funding, liquidations)	Not started; needs Envio or trade history
6	Wallet explorer, watchlist, compare, trader statistics	Unblocked by address registry
7	Market-making bot: testnet, then mainnet with hard limits and Telegram	Not started; needs funded wallet from you
8	Order form with risk preview, funding cost tracker	Not started
9	Liquidation-aware orders, zone-aware bot	Not started
10	Envio indexer, remaining correctness checks, logo, README, GitHub share, four videos, submission	Not started

Schedule. Three days remain to the 13 October deadline, and the bot has still not started. It is the one piece that needs days of real activity and your funding step. I recommend we do the bot on testnet immediately after Step 3.